

Week 9: Graph Signal Processing

Doing Signal Processing on Power-Grid Networks Instead of a
Regular Time Axis

Instructor / Mentor

Kiki Research Training Program

Week 9

Contents

1. Class Mission
2. Graph Theory Review
3. From Power Grids to Graph Signals
4. Graph Laplacian Intuition
5. Graph Frequency and FFT Connection
6. Power-Grid Applications
7. Python and PyGSP Mini Lab
8. Wrap-Up

Class Mission

Week 9 Mission

Core idea

Graph Signal Processing (GSP) means doing signal processing on data that lives on the **nodes of a graph** instead of on a regular time axis.

1. Review graph theory: nodes, edges, adjacency matrices, weights, and degrees.
2. Turn a power network into a graph with measurements attached to nodes.
3. Understand the graph Laplacian $L = D - A$ as a neighbor-comparison machine.
4. Connect FFT intuition to graph frequency: smooth changes vs. sharp changes across a network.
5. Use Python and PyGSP to visualize, smooth, and inspect graph signals.

Today's promise

By the end, students should be able to say: “This sensor value is strange because it disagrees with its network neighbors.”

120-Minute Flow

1. **0–15 min:** graph theory speed review.
2. **15–30 min:** power systems as graphs.
3. **30–50 min:** graph signals: values living on nodes.
4. **50–70 min:** graph Laplacian intuition: $L = D - A$.
5. **70–85 min:** graph frequency and connection to FFT.
6. **85–110 min:** Python / PyGSP mini lab.
7. **110–120 min:** applications, exit ticket, and homework.

Where This Fits in the Course

You already know

- Time-domain signals
- Frequency-domain intuition
- FFT as a pattern-finding tool
- Nodes and edges
- Adjacency matrices
- Weighted graphs and degrees

Today we combine them

- Signal processing asks: “How does a value change?”
- Graph theory asks: “Who is connected to whom?”
- GSP asks: “How does a value change **across a network?**”

Graph Theory Review

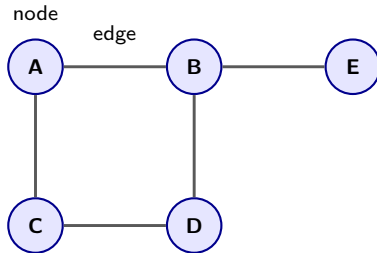
Graph Review: Nodes and Edges

Graph vocabulary

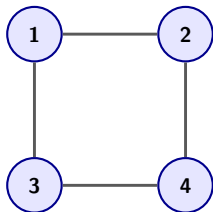
- A **node** is an object or location.
- An **edge** is a relationship or connection.
- A graph is a map of which objects are neighbors.

Power-grid translation

Nodes can be power stations, substations, buses, or sensors. Edges can be transmission lines.



Graph Review: Adjacency Matrix



Adjacency matrix A

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

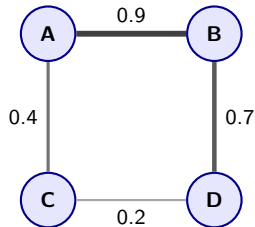
- $A_{ij} = 1$ means node i and node j are connected.
- $A_{ij} = 0$ means they are not direct neighbors.

Graph Review: Weighted Edges

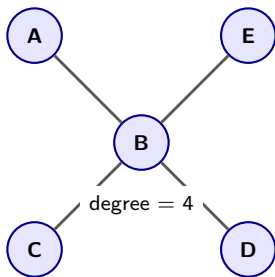
Not all connections are equal

A weighted graph stores the **strength** of each connection.

- Higher weight: stronger coupling or shorter electrical distance.
- Lower weight: weaker connection or longer electrical distance.
- In a power grid, a line weight might describe capacity, admittance, or similarity.



Graph Review: Degree



Degree means “how many neighbors”

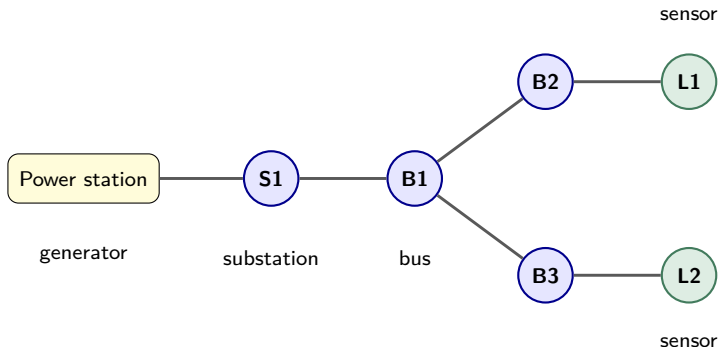
d_i = number of edges touching node i

Weighted degree

In a weighted graph, degree is often the **sum of connection weights** touching a node.

From Power Grids to Graph Signals

Power System as a Graph



Graph model

The physical grid becomes a graph: important locations are nodes, and electrical connections are edges.

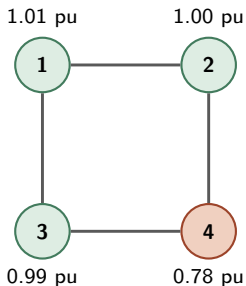
What Is a Graph Signal?

Definition

A **graph signal** is one number attached to each node of a graph.

Power-grid graph signals

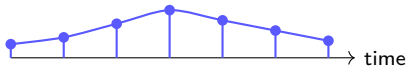
- Voltage magnitude at each bus
- Current on each monitored location
- Frequency deviation at each sensor
- Load at each neighborhood
- Transformer temperature



Time Signal vs. Graph Signal

Normal signal processing

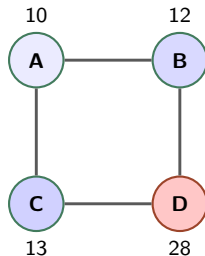
Data lives on a regular time axis.



Question: how does the signal change over time?

Graph signal processing

Data lives on connected nodes.



Question: how does the signal change across neighbors?

The Main GSP Question

GSP asks neighbor-aware questions

If two nodes are strongly connected, should their measurements usually be similar, different, or related in a predictable way?

Smooth network pattern

Nearby nodes have similar values.

- Normal voltage profile
- Slowly changing temperature
- Neighborhood load levels

Sharp network pattern

Connected nodes suddenly disagree.

- Bad sensor
- Local fault
- Sudden load spike
- Communication error

Graph Laplacian Intuition

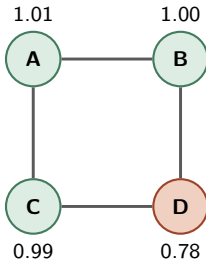
Why We Need the Graph Laplacian

In time signals

We can compare a value to nearby time samples: “Is today very different from yesterday?”

In graph signals

We compare a node to its graph neighbors: “Is this bus very different from connected buses?”



Node D disagrees with its neighbors.

The Three Matrices: A , D , and L

A : adjacency

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

Who is connected?

D : degree

$$D = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix}$$

How many neighbors?

$L = D - A$

$$L = \begin{bmatrix} 2 & -1 & -1 & 0 \\ -1 & 2 & 0 & -1 \\ -1 & 0 & 2 & -1 \\ 0 & -1 & -1 & 2 \end{bmatrix}$$

Compare with neighbors.

Laplacian as a Neighbor-Comparison Machine

Intuition, not heavy algebra

When L acts on a graph signal x , it asks each node:

How different am I from my neighbors?

For one node

If node i has two neighbors j and k , then roughly:

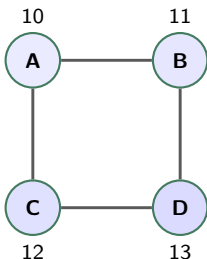
$$(Lx)_i = 2x_i - x_j - x_k$$

Meaning

- Near zero: node agrees with neighbors.
- Large positive or negative: node stands out.
- This is useful for detecting local surprises.

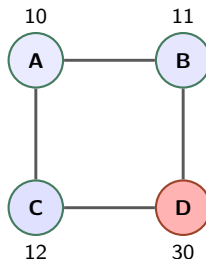
Visual Example: Smooth vs. Spiky Graph Signal

Smooth signal



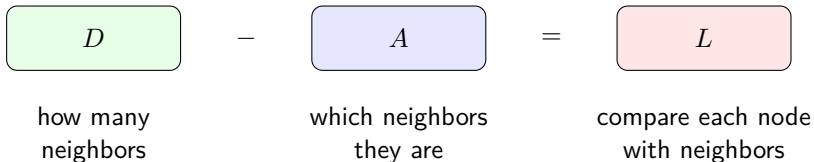
Low graph frequency: gentle changes across edges.

Spiky signal



High graph frequency: sharp changes across edges.

One-Sentence Meaning of $L = D - A$



Plain English

The graph Laplacian turns topology into a tool for measuring local disagreement.

Graph Frequency and FFT Connection

What Is Graph Frequency?

Graph frequency is about change across connected nodes

A graph signal has **low graph frequency** if connected nodes have similar values. It has **high graph frequency** if connected nodes often have very different values.

Low graph frequency

Smooth voltage profile, normal temperature field, gradual load pattern.

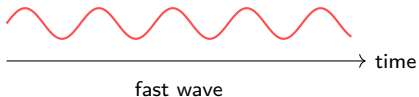
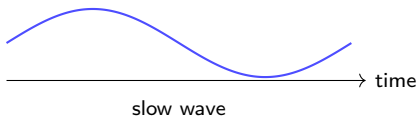
High graph frequency

Bad sensor reading, local fault, sudden abnormal load, disconnected behavior.

Connecting to FFT Knowledge

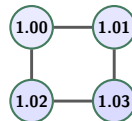
Fourier analysis in time

Finds slow and fast changes over time.



Fourier thinking on graphs

Finds smooth and sharp changes over a network.



smooth
pattern



sharp
pattern

Normal FFT vs. GSP: Same Spirit, Different Domain

Question	Time signal / FFT	Graph signal / GSP
Where does data live?	On regular time samples	On graph nodes
What is nearby?	Previous and next time points	Connected neighbor nodes
What is low frequency?	Slow change over time	Smooth change across edges
What is high frequency?	Rapid oscillation over time	Sharp disagreement across edges
Example	60 Hz voltage waveform	Voltage anomaly at one bus

Graph Fourier Transform: Conceptual Version

No heavy math today

A Graph Fourier Transform is like asking: “How much of this graph signal is smooth network pattern, and how much is sharp network pattern?”

Low graph-frequency ingredient

Values change gently from node to node.

Middle ingredient

Some regions differ from other regions.

High graph-frequency ingredient

Neighboring nodes disagree sharply.

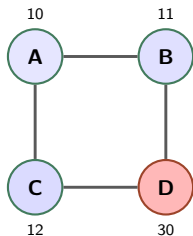


Low-Pass Graph Filtering

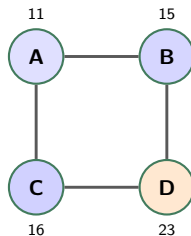
Filtering idea

A graph low-pass filter keeps smooth network patterns and reduces sharp local spikes.

Before filtering



After smoothing



Important caution

Smoothing can clean noise, but it can also hide real faults. Engineers must interpret filtered results carefully.

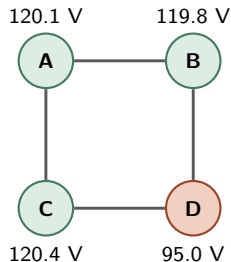
Power-Grid Applications

Application 1: Abnormal Sensor Detection

Situation

A voltage sensor reports a value far away from nearby connected sensors.

- Maybe the sensor is broken.
- Maybe there is a local power-quality issue.
- Maybe the grid topology changed.



GSP idea

Use neighbor disagreement to flag nodes that deserve human investigation.

Application 2: Smoothing Noisy Measurements

Situation

Sensors are not perfect. Some values jump because of measurement noise, communication errors, or temporary interference.

Before GSP

Treat every sensor independently. Noise at one sensor can look like a real event.

With GSP

Use the network structure: if neighbors agree, a strange isolated value may be noise.

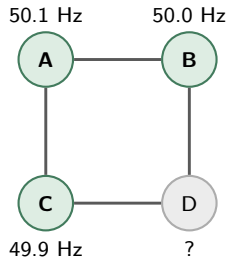
Raw sensor map $\rightarrow$ neighbor-aware cleaned map

Application 3: Filling Missing Node Values

Question

If one sensor fails, can we estimate its missing value using connected neighbors?

- Nearby nodes often carry useful clues.
- The graph tells us which clues are most relevant.
- Smoothness assumptions can help fill gaps.



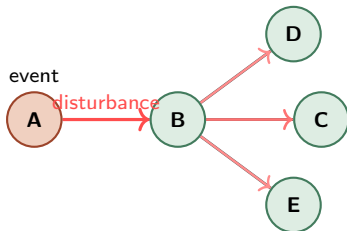
Simple first guess

Estimate a missing node by averaging its connected neighbors, then improve with more advanced GSP methods later.

Application 4: Understanding Spread Through a Network

Signals can spread

A disturbance at one location can influence connected locations, especially when the physical system is tightly coupled.



Research mindset

GSP helps researchers connect data patterns to the network paths through which physical effects may travel.

Python and PyGSP Mini Lab

PyGSP Demo 1: Build a Small Power Graph

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from pygsp import graphs
4
5 # 6-node mini grid: generator, substations, buses, sensors
6 A = np.array([
7     [0, 1, 1, 0, 0, 0],
8     [1, 0, 1, 1, 0, 0],
9     [1, 1, 0, 0, 1, 0],
10    [0, 1, 0, 0, 1, 1],
11    [0, 0, 1, 1, 0, 1],
12    [0, 0, 0, 1, 1, 0],
13 ], dtype=float)
14
15 G = graphs.Graph(A)
16 G.set_coordinates(np.array([
17     [0, 1], [1, 1.6], [1, 0.4],
18     [2, 1.6], [2, 0.4], [3, 1],
19 ]))
```

Teaching note

The matrix A stores the network. PyGSP turns that matrix into a graph object we can plot and process.

PyGSP Demo 2: Attach a Graph Signal

```
1 # Example graph signal: voltage magnitude at each node, in per unit
2 voltage = np.array([1.01, 1.00, 1.02, 0.99, 1.01, 0.82])
3
4 plt.figure(figsize=(6, 4))
5 G.plot_signal(voltage, vertex_size=700, colorbar=True)
6 plt.title("Voltage as a graph signal")
7 plt.show()
```

What to look for

Most nodes are near 1.0 per unit. Node 6 is much lower.

GSP question

Is node 6 abnormal compared with its graph neighbors?

PyGSP Demo 3: Use the Laplacian for Disagreement

```
1 # PyGSP stores the graph Laplacian as G.L
2 L = G.L
3
4 # Large magnitude means a node disagrees with its neighbors
5 disagreement = L @ voltage
6
7 for node_id, score in enumerate(disagreement):
8     print(node_id, round(score, 3))
9
10 plt.figure(figsize=(6, 4))
11 G.plot_signal(np.abs(disagreement), vertex_size=700, colorbar=True)
12 plt.title("Neighbor disagreement score")
13 plt.show()
```

Interpretation

This is not a final fault detector. It is an interpretable first clue: high disagreement means “please investigate.”

PyGSP Demo 4: Graph Fourier Transform Concept

```
1 # Conceptual step: decompose the node signal into graph-frequency patterns.
2 # Low graph-frequency patterns are smooth across edges.
3 # High graph-frequency patterns change sharply across edges.
4
5 G.compute_fourier_basis()
6 voltage_hat = G.gft(voltage)
7
8 plt.figure(figsize=(6, 3))
9 plt.stem(np.arange(len(voltage_hat)), np.abs(voltage_hat))
10 plt.xlabel("Graph-frequency index")
11 plt.ylabel("Magnitude")
12 plt.title("Conceptual graph-frequency recipe")
13 plt.grid(True)
14 plt.show()
```

No eigenvalue details today

For now, read this plot like an FFT-style recipe: more energy at higher graph-frequency indices means more sharp network disagreement.

PyGSP Demo 5: Simple Graph Smoothing Filter

```
1 # A simple low-pass graph filter: move each node slightly toward neighbors.
2 # alpha must be small; this is a gentle one-step smoothing operation.
3 alpha = 0.12
4 voltage_smooth = voltage - alpha * (G.L @ voltage)
5
6 plt.figure(figsize=(10, 4))
7 plt.subplot(1, 2, 1)
8 G.plot_signal(voltage, vertex_size=600, colorbar=True)
9 plt.title("Original voltage")
10
11 plt.subplot(1, 2, 2)
12 G.plot_signal(voltage_smooth, vertex_size=600, colorbar=True)
13 plt.title("Smoothed voltage")
14 plt.show()
```

Meaning

The filter reduces isolated spikes by using the graph as a guide for what “nearby” means.

Mini Lab Prompts

1. Change node 6 from 0.82 to 0.98. What happens to the disagreement score?
2. Change the graph by removing one edge connected to node 6. Does the anomaly look stronger or weaker?
3. Add random noise to all nodes. Does smoothing make the map easier to interpret?
4. Create a missing value at one node and estimate it using the average of its neighbors.
5. Explain one risk of blindly smoothing grid measurements.

Lab report sentence

“GSP helped me use the network structure to compare _____ with _____.”

Common GSP Mistakes

Mistake

- Treating node order like time order.
- Forgetting that topology changes the answer.
- Smoothing away a real fault.
- Thinking every outlier is a bad sensor.
- Ignoring physical units.

Fix

- Use edges to define neighborhood.
- Check whether the graph model is realistic.
- Compare raw and filtered signals.
- Investigate before concluding.
- Explain what each node value means physically.

Wrap-Up

Summary: What GSP Is Useful For

One-sentence summary

GSP is signal processing for data that lives on connected nodes, such as measurements across a power grid.

Useful for

- Detecting abnormal sensors in a power grid
- Smoothing noisy measurements
- Filling in missing node values
- Understanding how signals spread through a network

Big intuition

FFT finds slow and fast changes over time.
GSP finds smooth and sharp changes over a network.

Exit Ticket

1. What is a graph signal?
2. In a power grid, what could be a node? What could be an edge?
3. What does the graph Laplacian compare?
4. What does high graph frequency mean in plain English?
5. Name one way GSP could help a smart-grid researcher.

Final phrase to remember

GSP lets us analyze **measurements plus relationships**, not just measurements alone.

Thank you!

Next: using GSP ideas to support anomaly detection and smarter grid decisions.